

Informe técnico - Pipeline ML End-to-End con MLOps

Tarea 2: Implementación de Pipeline ML End-to-End

Autor: Julio Rios

Repositorio: <https://github.com/Jrios68/ml-pipeline-e2e>

1. Resumen ejecutivo

El proyecto implementa un pipeline completo de Machine Learning orientado a resolver un caso de priorización analítica de clientes. La versión final procesa múltiples archivos de extracción, denominados p1_extrac.csv a p10_extrac.csv, ubicados en la carpeta data/raw/. El flujo consolida los archivos, ejecuta preprocesamiento, entrenamiento, evaluación, registro del modelo, monitoreo, postprocesamiento TLV, generación de archivos de réplica, exposición mediante API y visualización en dashboard. La solución aplica principios de MLOps para asegurar reproducibilidad, trazabilidad y preparación para ambientes productivos.

2. Objetivo del pipeline

El objetivo es transformar datos históricos en una herramienta de decisión que permita priorizar acciones de negocio. El modelo estima una probabilidad de ocurrencia del evento objetivo y luego combina dicha probabilidad con variables de valor y frescura para construir una puntuación TLV. Esta puntuación permite ordenar los clientes y asignarlos a grupos de ejecución.

3. Arquitectura del proyecto

Componente	Propósito
main.py	Orquesta la ejecución completa del pipeline.
preprocessing.py	Lee p1_extrac.csv a p10_extrac.csv, consolida datos, crea p_codmes y genera train/test/validation.
training.py	Entrena XGBoost, optimiza hiperparámetros con Optuna y registra el modelo en MLflow.
monitoring.py	Calcula PSI, AUC, recall y genera archivos de monitoreo.
postprocessing.py	Calcula TLV, segmenta clientes y genera archivos de réplica.
api.py	Expone el modelo mediante FastAPI para inferencia individual y por lote.
dashboard.py	Visualiza resultados y métricas mediante Streamlit.

4. Flujo End-to-End

El pipeline se ejecuta desde main.py con el comando `python main.py --input "data/raw" --n-trials 5`. El flujo realiza: 1) lectura de los archivos p*_extrac.csv; 2) consolidación en un único DataFrame; 3) estandarización de columnas como tip_doc a tipdoc; 4) creación de p_codmes desde partition o p_fecinformacion; 5) limpieza e imputación; 6) entrenamiento y optimización; 7) registro en MLflow; 8) monitoreo; 9) postprocesamiento TLV; 10) réplica; 11) API; y 12) dashboard.

5. Trazabilidad y versionado

El proyecto registra los experimentos en MLflow, incluyendo parámetros, métricas y artefactos del modelo. Además, guarda una versión local del modelo en models/model.pkl y la lista de variables utilizadas en models/feature_names.json. La metadata del preprocesamiento se conserva en data/processed/preprocessing_metadata.json, permitiendo consistencia entre entrenamiento e inferencia. La ejecución final validada generó el modelo cu_venta_xgb, versión 3, con Run ID cd6aa30e475548b5a11ee8ce19190566.

6. Evaluación y monitoreo

Métrica	Uso
AUC	Mide la capacidad del modelo para discriminar entre clases.
Recall	Evalúa la capacidad de capturar casos positivos.
Recall por decil	Mide la concentración de casos positivos en los grupos con mayor score.
PSI	Detecta deriva entre la distribución de scores de entrenamiento y validación.

Los umbrales aplicados para PSI son: menor a 0.10 = OK, entre 0.10 y 0.25 = advertencia, y mayor a 0.25 = alerta de deriva severa.

7. API de inferencia

La API se implementa con FastAPI y permite ejecutar inferencias individuales o por lote. Los endpoints principales son /health, /predict y /predict-batch. La API carga el modelo entrenado, estandariza entradas como partition y tip_doc, y alinea las variables recibidas con la estructura utilizada durante el entrenamiento.

8. Dashboard

El dashboard se implementa con Streamlit y permite revisar la distribución de grupos TLV, los clientes con mayor puntuación, las métricas de monitoreo, el recall acumulado por decil y la distribución de scores. Esta vista facilita la comunicación de resultados a usuarios técnicos y de negocio.

9. Valor de negocio

La solución permite priorizar clientes según probabilidad estimada y valor esperado, reduciendo el esfuerzo manual de análisis y enfocando los recursos en los segmentos con mayor impacto. Para la organización, esto se traduce en eficiencia operativa, trazabilidad de decisiones, reducción de tiempos de análisis, mejor gobierno del modelo y capacidad de escalar la solución hacia un entorno productivo.

10. Reproducibilidad y ejecución

La reproducibilidad se garantiza mediante una estructura modular, archivo requirements.txt, semilla de entrenamiento, salidas estandarizadas y documentación en README.md e INSTRUCCIONES_EJECUCION_WINDOWS.md. La ejecución completa se realiza con `python main.py --input "data/raw" --n-trials 5`. Los experimentos se revisan con mlflow ui, la API se inicia con `uvicorn api:app --reload` y el dashboard con `streamlit run dashboard.py`.

11. Conclusiones

El pipeline desarrollado cumple con los requerimientos de una solución ML End-to-End: automatización, versionado, monitoreo, inferencia y comunicación ejecutiva del impacto. Además, queda preparado para reutilizarse con nuevas particiones de datos y para evolucionar hacia despliegues productivos con mayor control de datos, seguridad, CI/CD y monitoreo continuo.